Flarre-Fuzz

An adversarial safety layer that attacks AI-written insurance code before it reaches a carrier's systems

Submitted for: Flarre AI Innovation Challenge 2026 | **Name:** [Your Full Name as per Aadhaar] | **College:** [Your College Name]

1. Problem statement

AI agents now write production code for insurance platforms, including premium and rating calculations. A small mistake in this code, such as a wrong age boundary, a missing cap or a rounding error, can quietly misprice thousands of policies, causing financial loss and regulatory exposure. Today, such code is validated mainly through human-written test cases. This is slow, and it tends to cover the cases people already expect, while the failures that matter most hide in rare and extreme inputs.

2. Impact and relevance

- **Financial:** pricing errors scale with every policy the code touches.
- **Regulatory:** insurers must apply approved rates and rules correctly; errors invite audit findings.
- **Delivery speed:** manual validation is a bottleneck for teams shipping AI-generated code.
- **Trust:** enterprise buyers need evidence of safety and auditability before adopting AI-written code. This matches Flarre's stated focus on governance, auditability and safety.

3. Proposed solution

Flarre-Fuzz is an automated testing pipeline in which different components play different roles, with a human making the final decision:

Component	Type	Role
Bully	LLM + code	Generates attack inputs in three layers: boundary values, random fuzzing, and rare customer profiles invented by an LLM. Every LLM-generated profile is validated by code before use.
Sandbox	Code	Runs the code under test in an isolated subprocess with a timeout, capturing crashes. Stand-in for a system clone.
Referee	Deterministic code	Judges every result using (a) invariants that must always hold and (b) comparison with a trusted reference calculation. Deliberately not an LLM, so verdicts are repeatable.
Fixer	LLM	Explains failures and proposes a patch.
Re-verify	Deterministic code	Re-runs the full suite on the patched code (max 3 attempts) to confirm the bugs are gone and nothing new broke.
Human + audit log	Person + code	Reviewer approves or rejects the fix; each step is logged with a timestamp.

Design principle: LLMs handle creativity and explanation; deterministic code handles correctness. This avoids relying on a language model to judge its own work.

4. Architecture

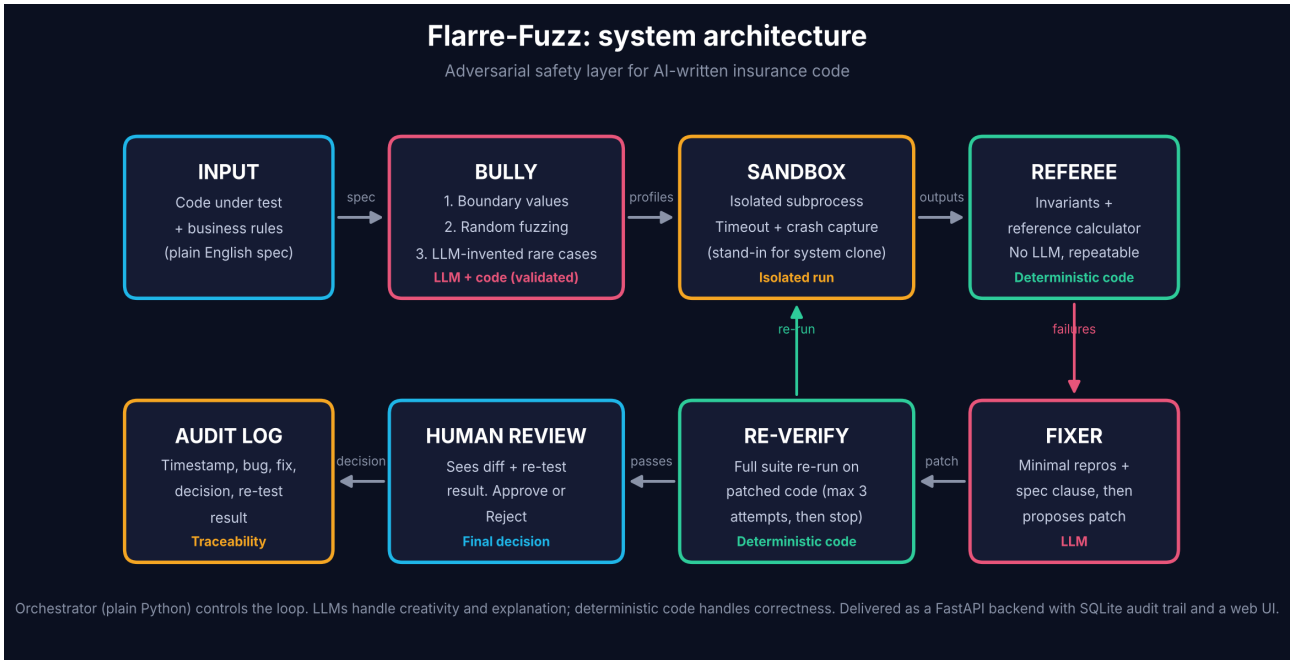


Figure 1. Flarre-Fuzz architecture. A plain-Python orchestrator controls the loop and the retry limit.

5. AI utilisation

- **Generative AI (Bully):** invents rare, adversarial customer profiles from the written business rules.
- **Generative AI (Fixer):** diagnoses failures and drafts patches with plain-English explanations.
- **Orchestrated multi-component workflow:** structured JSON hand-offs, retry limits, and a human approval gate.
- **Guardrails:** LLM output is validated by code, and all correctness decisions are deterministic.

6. Prototype and results

A working prototype was built as a FastAPI backend with a web dashboard and a 17-test pytest suite. The code under test is a simplified personal-auto premium calculator written for this project, with **five deliberately planted bugs**: an age-boundary error, a missing claims surcharge cap, a missing discount cap, truncation instead of rounding, and an off-by-one at age 29 in a mid-range surcharge rule.

Planted bug	Boundary only	Random only	Boundary + random
Age boundary (<= instead of <)	found (24)	found (11)	found (33)
Missing claims surcharge cap	found (137)	found (777)	found (904)
Missing 20% discount cap	found (108)	found (263)	found (362)
Truncation instead of rounding	found (89)	found (313)	found (392)
Age 29 off-by-one (mid-range rule)	**MISSED**	found (8)	found (8)
False alarms on correct code	0	0	0
Profiles generated	216	935	1,126

All five bugs are detected by the combined deterministic layers, and the corrected implementation produces no failures. The ablation shows why several layers are needed: the hand-picked boundary

layer never probes age 29 and misses that bug, while random fuzzing finds it. The LLM layers are built and validated but have **not yet been measured**. Figures are from a toy calculator, not a production system.

7. Implementation approach

- **Stack:** Python, FastAPI, Pydantic, SQLite (audit trail), a web UI served by the API, subprocess isolation, optional LLM API.
- **Algorithms:** hash-map neighbour checks for the monotonic invariant, delta-debugging shrinker for minimal reproductions, TF-IDF retrieval linking each failure to the spec clause it violates.
- **Phase 1 (done):** calculator with planted bugs, reference, Referee, Bully (boundary + random), sandbox, re-verification.
- **Phase 2:** LLM-based Bully and Fixer evaluated across multiple runs; patch quality measured.
- **Phase 3:** support multiple rule sets and languages; connect to a real rating engine's test environment.

8. Scalability and next steps

- Add new rule sets by supplying a spec and a reference implementation, with no change to the pipeline.
- Swap the subprocess sandbox for container isolation in production.
- Use workflow tooling for durable retries, and connectors to plug into an existing testing agent.

9. Limitations (stated honestly)

- Testing can find bugs but cannot prove their absence; the system reduces risk, it does not guarantee safety.
- The Referee is only as good as its invariants and reference; a wrong reference gives wrong verdicts.
- LLM-proposed patches can be wrong; hence full re-verification and human approval before acceptance.
- The prototype uses a simplified calculator and a subprocess sandbox. It is not a replica of a carrier's core system.
- Flarre's platform already includes a Testing Agent. Flarre-Fuzz is designed as a complementary adversarial layer, and its internals were not available for this design.

10. Expected benefits

- Broader edge-case coverage than hand-written tests, in minutes instead of days.
- Each failure is traced to a specific business rule, which makes review faster.
- A full audit trail for every automated decision and human approval.
- Human engineers are freed to review fixes instead of hunting for edge cases.